

NumPy I: arrays and shapes

Tuesday ended at a wall: `2 * xs` repeated the list instead of doubling it. Today that line starts working, your grid becomes a real two-dimensional object, and you meet `.shape` — which from now until December is the first thing you print whenever anything goes wrong.

Unit 1 • Foundations

80 minutes

Lab 3 • predict every shape

Laptops open

[Where we are](#) [The import](#) [Shape](#) [Two dimensions](#) [Name the operation](#) [Which axis](#) [Slices are windows](#) [Lab 3](#) [Before Tuesday](#)

Where we are

Two sessions of Python behind you. You can write a function, walk a list with an accumulator, slice, and build a grid out of nested lists. That is genuinely everything the language side of this course needs.

You also hit a wall on Tuesday, and it was not an accident — it was the last five minutes of the session by design. A list is a general-purpose container: it will hold numbers, text, other lists, all at once. Because it cannot assume its contents are numbers, `*` and `+` mean the only things that make sense for any container at all — repeat and join. Arithmetic on every element is not something a list is allowed to guess at.

TODAY, IN ONE SENTENCE

An **array** is a container that *does* assume every element is a number of the same kind. That single assumption buys you three things: arithmetic that means what you want, two-dimensional indexing that reads like mathematics, and a hundredfold speed-up. Everything else today is consequences of it.

PART 1 • 10 MINUTES

One import, and the wall comes down

Everything that failed on Tuesday now works. Then we ask what changed, because the answer explains the rest of the semester.

1

the first cell of every notebook from here on

PYTHON

```
import numpy as np

a = np.array([10, 20, 30, 40, 50, 60])

2 * a      → array([ 20, 40, 60, 80, 100, 120])  # doubled. finally.
a + 10     → array([20, 30, 40, 50, 60, 70])
a ** 2     → array([ 100, 400, 900, 1600, 2500, 3600])
a[2:4]     → array([30, 40])  # slicing: identical to Tuesday
```

Note the last line. Slicing did not change. Everything you learned about fenceposts on Tuesday transfers unmodified — which is why we spent ten minutes on it.

What actually changed

A Python list can hold anything: a number, a string, another list, all at once. That generality is why `*` could only mean "repeat" — the list has no idea whether arithmetic is meaningful for its contents.

A NumPy array gives up that generality in exchange for three things:



One type for everything

Every element is the same type — the `dtype`. Usually `float64` in this course. The array knows arithmetic is meaningful because it knows what it holds.



One contiguous block of memory

A list holds pointers scattered across memory. An array is one unbroken run of bytes, which is what lets the processor work on many at once.



The loop moves into C

`a ** 2` still loops over every element — but in compiled code, not in Python. That single fact is the ~100× you are about to measure.

the `dtype` is worth watching

PYTHON

```
a = np.array([1, 2, 3])
a.dtype      → dtype('int64')      # whole numbers in, integers out
(a / 2).dtype → dtype('float64')    # division always promotes
a.astype(float).dtype → dtype('float64') # force it when you need to

# making arrays without a list
np.zeros((2, 3))      # 2 rows, 3 columns, all 0.0
np.arange(0, 1, 0.25) → array([0. , 0.25, 0.5 , 0.75]) # stop excluded
np.linspace(-3, 3, 61) # 61 points, both ends included
```



Tuesday's line, replaced

You wrote `xs = [i/10 for i in range(-30, 31)]` and I said the workaround would disappear. It has: `np.linspace(-3, 3, 61)`. Say what you want — start, stop, how many — instead of describing how to count there.

Shape is the thing you debug

2

For the next twenty sessions, when something breaks, the first question is always the same: what shape is it?

a 2-D array, and its vital statistics

PYTHON

```
X = np.arange(12).reshape(3, 4)
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
X.shape    → (3, 4)    # a TUPLE – Tuesday's tuples, doing real work
X.ndim     → 2         # how many dimensions
X.size     → 12        # how many numbers in total
```

Read `(3, 4)` as **3 rows, 4 columns** — and in this course, almost always, **3 examples with 4 features each**. That convention holds from here to session 26. In session 23 you will handle images shaped `(60000, 784)`: sixty thousand examples, seven hundred and eighty-four pixels each.

Slicing gains a second dimension — and nothing else is new

one slice per dimension, separated by a comma

PYTHON

```
X[0:2, 1:3] # rows 0-2, columns 1-3
→ array([[1, 2],
       [5, 6]])
```

```
X[:, 0]     → array([0, 4, 8])      # every row, column 0      shape (3,)
X[1, :]     → array([4, 5, 6, 7])   # row 1, every column    shape (4,)
X[:, 0:1]   # every row, columns 0-1      shape (3, 1)
```



Look hard at the last two lines

`X[:, 0]` has shape `(3,)`. `X[:, 0:1]` has shape `(3, 1)`. Same six numbers, different shape. An index removes a dimension; a slice keeps it.

That distinction looks pedantic right now. In eleven minutes it will silently give you a wrong answer with no error message, and I want you to have seen it first.

RESHAPE, AND THE `-1` THAT MEANS "WORK IT OUT"

reshape

PYTHON

```
X = np.arange(12)

X.reshape(3, 4).shape    → (3, 4)
X.reshape(3, -1).shape   → (3, 4)  # "3 rows, you figure out the columns"
X.reshape(-1, 2).shape   → (6, 2)   # "2 columns, you figure out the rows"
```

You may use `-1` in exactly one position. It is not a magic value; it means "solve for this one so the total stays 12". You will write `.reshape(-1, 1)` more often than any other reshape in this course.

PART 3 • 12 MINUTES

Two dimensions, and the comma that replaces two brackets

Tuesday's grid, done properly. This is the notation the rest of the course is written in.

A nested list becomes a two-dimensional array in one call, and the moment it does, the indexing changes.

the same grid, before and after

PYTHON

```
import numpy as np

grid = [[4, 7, 2],
        [9, 1, 8],
        [3, 6, 5]]

a = np.array(grid)

grid[1][0]    # 9    the list way – two separate lookups
a[1, 0]       # 9    the array way – one lookup, two indices
a.shape       # (3, 3) three rows, three columns
```

The comma is not cosmetic. `grid[1][0]` fetches row 1 as a whole list and then indexes into it; `a[1, 0]` asks for one element by its position in two dimensions, which is why the next thing works and the list version does not.

A whole row, and a whole column

the colon means "everything along this axis"

PYTHON

```
a[1, :]       # [9, 1, 8]    row 1, every column
a[1]          # [9, 1, 8]    the same thing, shorter
a[:, 0]       # [4, 9, 3]    every row, column 0 ← try this with a list
a[:, 1:]      # the last two columns of every row
a[0:2, 0:2]   # the top-left 2x2 corner
```



`a[:, 0]` is the line to remember

Getting a whole column out of a list of lists takes a loop. Here it takes four characters. Read it left to right as "every row, column zero". When you meet a colour image in session 23, `img[:, :, 0]` will mean "every row, every column, the red channel" — the same idea with one more axis, and you will already know how to read it.

Slicing rules did not change

Everything from Tuesday still holds: half-open, so `0:2` gives two rows; negative indices count from the end; and the length of a slice is stop minus start. The only new thing is that you can now do it along both axes at once, separated by a comma.

predict each shape before running – this is Lab 3 in miniature

PYTHON

```
a.shape      # (3, 3)
a[0].shape   # (3,)   one row – one dimension, not two
a[:, 0].shape # (3,)   one column – also one dimension
a[0:1].shape # (1, 3)  a slice keeps the dimension; an index removes it
a[1, 1].shape # ()     a single number has no shape at all
```

The distinction that will cost you an hour in session 13

`a[0]` has shape `(3,)` but `a[0:1]` has shape `(1, 3)`. Indexing with a number *removes* that axis; slicing *keeps* it, even when the slice is one element long. Both contain the same three numbers. They will not multiply the same way, and the error message you get will be about shapes rather than about this.

Building arrays without typing them

PYTHON

```
np.zeros((2, 3))      # 2x3 of 0.0 – note the shape is one argument, a tuple
np.ones((3, 3))
np.arange(6)          # [0 1 2 3 4 5]   like range, but an array
np.arange(6).reshape(2, 3)
                        # [[0 1 2]
                        #    [3 4 5]]   same six numbers, different shape
```

`reshape` does not move any numbers; it changes how the same six values are addressed. That is why `reshape(2, 3)` works and `reshape(2, 4)` raises an error – six numbers cannot be arranged into eight slots, and NumPy tells you so rather than guessing.



Do this now – two minutes

Make `b = np.arange(12).reshape(3, 4)`. Without running anything, write down what these five give: `b.shape`, `b[2, 3]`, `b[:, 1]`, `b[1:, 2:]`, `b[0].shape`. Then check. Anything you got wrong is worth two minutes now rather than twenty in session 13.

PART 4 • 10 MINUTES

Stop visiting elements. Name the operation.

The mental shift that makes everything after session 9 possible.

4

the same job, two ways of thinking

PYTHON

```
# thinking about elements – "visit each one and do this to it"
out = []
for x in xs:
    out.append(x * x)

# thinking about the whole array – "square it"
out = xs ** 2
```

THE HABIT TO BUILD STARTING TODAY

If you are writing `for` over the elements of an array, stop and ask what operation you are describing. There is almost always a name for it, and the named version is both shorter and roughly a hundred times faster.

This is not about elegance. Session 16 trains a network for ninety passes over a dataset. At loop speed that is an afternoon; vectorised it is a coffee. You will iterate on your model ten times as often, which is the actual reason your results will be better.

Measured on an ordinary laptop, one million numbers, each job done both ways:

Task	Python	NumPy	Speed-up
Square every element	57.3 ms	0.73 ms	78×
Sum all elements	36.4 ms	0.36 ms	100×
Dot product of two vectors	97.7 ms	0.60 ms	163×
Rescale to the range 0–1	139.3 ms	2.29 ms	61×
Count elements above 0.5	42.3 ms	0.77 ms	55×



A real mistake, made while preparing this table

The first version of the rescaling task was `[(x - min(xs)) / (max(xs) - min(xs)) for x in xs]`. It looks fine. It never finished, and I killed it after several minutes.

`min(xs)` is inside the loop, so it is recomputed for every one of a million elements — a million passes over a million elements. The comprehension hid an accidental one-trillion-operation job behind one readable line. Hoisting `min` and `max` out gave the 139 ms above.

Watch for work that does not depend on the loop variable sitting inside the loop.

NumPy sidesteps this entirely: `(xs - xs.min()) / (xs.max() - xs.min())` evaluates each once, because you named the operation instead of describing the visit.

PART 5 · 8 MINUTES

axis=0 or axis=1?

5

Everyone guesses wrong at first. One sentence fixes it permanently.

same array, three different sums

PYTHON

```
X = np.arange(12).reshape(3, 4)    # shape (3, 4)

X.sum()                → 66          # everything          shape ()
X.sum(axis=0)          → array([12, 15, 18, 21]) # shape (4,)
X.sum(axis=1)          → array([ 6, 22, 38])    # shape (3,)
```

THE SENTENCE THAT ENDS THE CONFUSION

`axis=k` is the axis that **disappears**.

`X` is `(3, 4)`. Sum with `axis=0` and the 3 vanishes, leaving `(4,)` — one number per column. Sum with `axis=1` and the 4 vanishes, leaving `(3,)` — one number per row. Do not ask "which direction does it sum"; ask "which number do I want gone".

In our convention rows are examples and columns are features, so `axis=0` gives you something per feature — the mean of each column is exactly what you need to standardise a dataset in session 20 — and `axis=1` gives you something per example, which is how you get one loss value per training case.

PART 6 • 4 MINUTES

6

A slice is a window, not a copy

One callback: Tuesday's aliasing trap, now worse

On Tuesday `b = a` made a second label on the same list. With arrays, a *slice* does it too:

a slice is a view onto the same memory

PYTHON

```
A = np.arange(6).reshape(2, 3)
row = A[0]          # a VIEW, not a copy
row[0] = 99
print(A)             → [[99  1  2]
                       [ 3  4  5]]  # A changed

B = np.arange(6).reshape(2, 3)
row = B[0].copy()    # an actual copy
row[0] = 99
print(B)             → [[0  1  2]
                       [3  4  5]]  # B untouched
```

Views are a feature, not a flaw — they are why slicing a 60,000-row dataset costs nothing. But when you intend to modify, use `.copy()` and mean it.

LAB 3 • 16 MINUTES IN CLASS

7

Predict every shape before you run the cell

Lab 3 is a separate handout — `CSC4315_Lab03_Arrays_And_Shapes.html`, or the printed copy. It is mostly not a coding lab: you write down what you expect each line to produce, then run it and mark yourself. That sounds like less work than writing code and is worth considerably more, because shape errors are the single thing that will slow you down between now and December.



Same rules

Finish in the room, hand up after three minutes, and nothing here is a prerequisite for Tuesday. Session 7 begins from the same starting point for everyone.

Before Tuesday

1. **Finish Lab 3**, especially the prediction column. The value is in having committed to an answer before seeing the truth.
2. **Re-do Lab 2's `row_totals` in NumPy** — it is now one line, `a.sum(axis=1)`. Run both versions on the same grid and confirm they agree. Then look at the two pieces of code side by side and notice how much of Tuesday's version was bookkeeping rather than mathematics.
3. **Print `.shape` of five things** you make up. Get into the habit now, while nothing depends on it.



Session 7 — Tuesday 29 September, 16:00

NumPy II: broadcasting, and plotting what you compute. Broadcasting is the rule that lets arrays of different shapes work together — it is what makes `x - x.mean()` legal and what makes half of all shape errors happen. Then Matplotlib, kept to the three plot types this course needs, so that from session 8 onward we can look at a curve instead of describing one.